The architecture we've explored—separating the Policy Enforcement Point (PEP), Policy Decision Point (PDP), Policy Administration Point (PAP), and Policy Information Point (PIP)—emerged to address precisely those challenges.

Here's why this separation matters:

- Maintainability: Policies can be updated independently of application code. When Carol updates the access rules for ACMEProject, developers don't need to deploy new builds or touch code at all.
- Consistency: The same policy can be applied consistently across multiple applications and services, ensuring uniform behavior and minimizing drift.
- Transparency: Authorization decisions can be traced back to specific policies and contextual data, which enhances auditability and trust.
- Scalability: The ability to distribute PDPs and entity caches allows systems to grow without encountering a single bottleneck or central point of failure.
- Security: Keeping access control logic distinct from business logic limits the blast radius of bugs, reduces attack surfaces, and supports defense-in-depth.

In short, separating these concerns transforms authorization from a code-level detail into a platform capability. It converts access control into something that can be reasoned about, tested, managed, and governed, just like any other critical part of the system.

4.6 From architecture to models

While authorization is ultimately a yes-or-no decision, it also represents the system-level expression of relationships, roles, responsibilities, and context. In this chapter, we explored what makes authorization distinct from authentication, introduced the PARC model, and examined the architecture that supports dynamic, policy-based access control. Whether decisions are centralized for consistency or distributed for performance, the pattern